

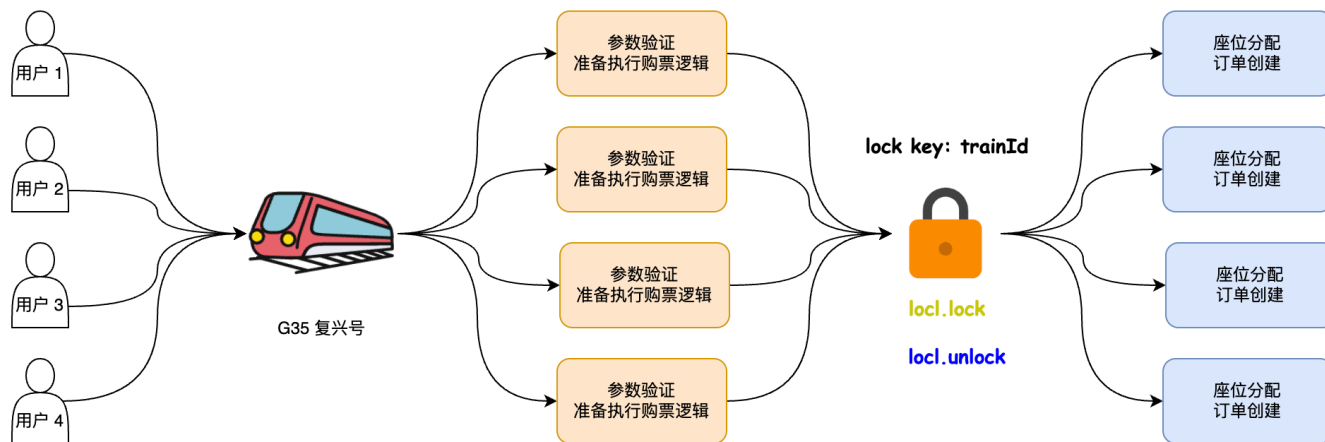
手把手之实现v2版本列车购票流程

阅读本文档之前，需要先阅读 v1 版本列车购票流程：[手把手实现列车购票流程](#)

v1 版本购票存在的问题

根据列车购票流程，也就是 v1 版本购票接口文档得知：为了保障列车座位不超卖以及一个座位不分配给多名用户，选择使用分布式锁来保障数据安全和一致性。

通过下图简单理解 v1 版本购票流程，我觉得存在几个较严重的问题。



瞬时高并发压垮系统

对于五一、国庆以及过年这些节日来说，一些热门列车的 TPS 少说有几十万 TPS。

什么是 TPS？

TPS (Transactions Per Second) 是一个衡量系统性能的指标，用于表示系统在单位时间内能够处理的事务数量。

在计算机领域中，TPS 通常用于度量一个系统或应用程序处理事务的能力和性能。一个事务可以是数据库的读写操作、网络请求、交易处理或其他类型的操作。

TPS 的值越高，表示系统能够在单位时间内处理更多的事务，即系统性能越好。通常，高性能的系统需要具备较高的 TPS 值，以满足对并发处理和快速响应的需求。

说了 TPS 的概念，捎带脚的说说什么是 QPS。

QPS (Queries Per Second) 是一个衡量系统负载或性能的指标，用于表示系统在单位时间内处理的查询请求数量。

在计算机领域中，QPS 通常用于度量数据库、网络服务器或其他系统的查询处理能力和性能。一个查询可以是数据库查询、网络请求、API调用或其他类型的查询操作。

QPS 的值越高，表示系统能够在单位时间内处理更多的查询请求，即系统的查询处理能力越强。高吞吐量和低延迟的系统通常需要具备较高的 QPS 值，以满足大量请求的处理需求。

需要注意的是，QPS 仅仅表示每秒处理的查询请求数量，而不考虑每个查询的复杂性、资源消耗和响应时间。

如果算上所有用户的 TPS，上千万也不是没有可能。如果按照 v1 版本的架构，让这些请求全部卡在获取分布式锁，这会造成什么问题？

众所周知，一个 SpringBoot 应用的同一时间在运行的请求是有限的，因为 SpringBoot 处理请求底层也是个线程池在做。我截图个 Hippo4j 监控到的 SpringBoot Tomcat 容器线程池举例。

别给马哥的开源动态线程池框架 [Hippo4j](#) 点 Star，求你们了。

The screenshot shows the Hippo4j 1.5.0 web interface. The left sidebar contains navigation links: 运行报表, 租户管理, 项目管理, 动态线程池, 容器线程池, Tomcat, Undertow, Jetty, 框架线程池, 线程池审核, 通知报警, 用户权限, 日志管理, 官网外链. The main content area shows the configuration for a dynamic thread pool. The 'Info' modal is open, displaying the following information:

负载信息					
当前负载	0%	峰值负载	1%	剩余内存	14.12GB
内存占比	Allocation: 109.80MB / Maximum available: 14.22GB				

线程信息					
核心线程	10	当前线程	2	最大线程	200
活跃线程	1	同存最大线程	2		

队列信息					
队列容量	2147483647	队列元素	0	队列剩余容量	2147483647
阻塞队列	TaskQueue				

其它信息					
任务总量	1	最后更新时间	2023-08-24 22:50:17	拒绝策略	RejectPolicy

Red arrows point to the 'Core Threads' (10) and 'Maximum Threads' (200) values in the 'Thread Information' section.

通过上图得知，SpringBoot Tomcat 容器默认情况下，同一时间最多能处理 200 个请求。如果要应对上千万的 TPS 明显是不可能的。

按照 v1 架构的设计存在以下问题：大量请求会因为分布式锁的申请而发生阻塞，导致请求无法快速处理。这会导致后续请求长时间被阻塞，使系统陷入假死状态。无论请求的数量有多大，系统都无法返回响应。此外，随着请求的积累，还存在内存溢出的风险。更糟糕的是，如果 SpringBoot Tomcat 的线程池被分布式锁占用，查询请求也将无法得到响应。

先买票不一定有票

咱们 v1 架构中使用分布式锁保障数据一致性以及其它功能，大家思考一个问题，分布式锁的解锁行为是顺序的么？代码如下所示：

```
RLock lock = redissonClient.getLock(lockKey);
lock.lock();
try {
    return executePurchaseTickets(requestParam);
} finally {
    lock.unlock();
}
```

假设我有三个人购票，张三、李四和王五，他们网络请求到达服务端的时间是依次，同名称排列顺序。

正常来说，可能是张三先获取分布式锁，购票后李四再获取分布式锁，紧接着才是王五。但是实际情况有可能张三释放锁后，后来的王五会获取到锁。这种情况就是因为默认 Redisson 的获取锁是非公平锁，这个概念和 ReentrantLock 中的非公平锁概念是一致的。

我再系统解释下什么是非公平锁。

非公平锁（Non-Fair Lock）是一种锁的获取策略，它允许在锁释放时，新的请求可以插队获取锁，而不必按照请求的顺序等待。相对而言，公平锁（Fair Lock）则按照请求的顺序分配锁，确保较早请求的线程先获得锁。

使用非公平锁可以带来一些性能上的优势，例如减少线程切换和提高吞吐量。然而，非公平锁也存在一些问题：

1. 饥饿问题（Starvation）：由于非公平锁允许新的请求插队获取锁，较早的请求可能会被后续请求一直抢占，导致某些线程长时间无法获取锁，导致饥饿问题。这可能会影响系统的公平性和可预测性。
2. 公平性问题：非公平锁的获取顺序不遵循请求的先后顺序，因此无法保证所有线程都能在合理的时间内获得锁，可能导致某些线程长时间等待。

为了避免这种情况，我们需要按照公平锁的方式请求和释放锁，使用后的场景就是释放锁后再获取锁要严格按照请求分布式锁的先后顺序执行。

分布式锁压力

通过上图得知，我们在真正进行作为分配和创建订单前，需要先获取到分布式锁，然后才能执行接下来的流程。

问题来了，假如一趟列车有几十万人抢票，但是真正能购票的用户可能也就几千人。也就意味着哪怕几十万人都去请求这个分布式锁，最终也就几十万人中的几千人是有效的，其它都是无效获取分布式锁的行为。

那这块的分布式锁逻辑是不是可以优化呢？比如不让所有抢购列车的用户去申请分布式锁，而是让少量用户去请求获取分布式锁。这样优化的话，可以极大情况节省 Redis 申请分布式锁的开销压力。

用户购票响应慢

咱们以高铁复兴号举例：一趟列车有三种座位，分别是商务座、一等座和二等座。v1 版本购票接口的分布式锁锁定规则是锁定整个列车，也就是列车的唯一键列车 ID。相当于同一时间一趟列车只允许单个用户进行分配座位、创建订单等流程。

假设 A 用户购买二等座、B 用户购买一等座以及 C 用户购买商务座，按照现有流程这三个用户的请求是串行的，但从真实场景以及数据一致性分析，这三个用户是不是也能并行？三个用户对应三个不同的座位，互不影响，完全可以并行。这样可以极大提升系统出票的能力，个别场景下，系统的处理能力可以提升 300%。

当然，事情往往不会进展的如此顺利。如果 A 用户购买了二等座和一等座，B 用户购买了一等座和商务座，这就涉及到一个锁重合互斥问题。

如何优雅解决？且看下文解析。

令牌限流算法

本章节解决了瞬时高并发压垮系统问题。

在这之前，先和大家聊一聊什么是限流？常见的限流算法有哪些？

什么是限流

限流（Rate Limiting）是一种应用程序或系统资源管理的策略，用于控制对某个服务、接口或功能的访问速率。它的主要目的是防止过度的请求或流量超过系统的处理能力，从而保护系统的稳定性、可靠性和安全性。

通过限制访问速率，限流可以防止以下问题的发生：

1. 过度使用资源：限流可以防止某个用户或客户端过度使用系统资源，从而保护服务器免受过载的影响。
2. 防止垃圾请求：限流可以过滤掉恶意或无效的请求，例如恶意攻击、爬虫或垃圾邮件发送等。
3. 维护服务质量：通过限制访问速率，可以确保每个请求都能够得到适当的处理和响应时间，从而提高服务质量和用户体验。
4. 控制成本：限流可以帮助控制系统资源的使用，避免因过多的请求而导致不必要的成本增加。

上文也讲到过，一个列车可能就几百上千人能购买成功，但可能会有远超过这个量级的用户进行抢票，在真正执行抢票逻辑前，可以通过限流算法进行限制，只让少量用户操作购票流程。

常见限流算法

限流可以通过多种算法方式实现，比如：

1. 固定窗口算法（Fixed Window Algorithm）：该算法将时间划分为固定大小的窗口，例如每秒、每分钟或每小时。在每个窗口内，限制请求的数量不得超过预设的阈值。这种算法简单直观，但可能存在突发请求超过阈值的问题。
2. 滑动窗口算法（Sliding Window Algorithm）：该算法将时间划分为连续的时间片段，例如每秒划分为多个小的时间片段。每个时间片段都有自己的请求计数器，记录在该时间片段内的请求数量。当请求到达时，会逐渐删除过时的时间片段，并根据当前时间片段内的请求数量判断是否超过阈值。这种算法可以更好地处理突发请求。
3. 令牌桶算法（Token Bucket Algorithm）：该算法模拟了一个令牌桶，桶中以固定速率生成令牌。每个令牌代表一个请求的许可。当请求到达时，需要先从令牌桶中获取令牌，如果桶中没有足够的令牌，则请求被限制。这种算法可以平滑地控制请求的速率。
4. 漏桶算法（Leaky Bucket Algorithm）：该算法类似于一个漏桶，请求以固定速率进入漏桶。如果漏桶已满，则多余的请求将被丢弃或延迟处理。这种算法可以稳定请求的处理速率，防止突发请求对系统造成压力。

这些算法都有不同的特点和适用场景，选择适合的限流算法取决于应用程序的需求和预期的限流效果。在实际应用中，也可以根据具体情况结合多种算法来实现更复杂的限流策略。

这些算法网上介绍的较为完善，大家可以搜索相关文章详细了解，这里不过多赘述。而咱们 12306 使用的限流方案却不是上面说的这几种之一。

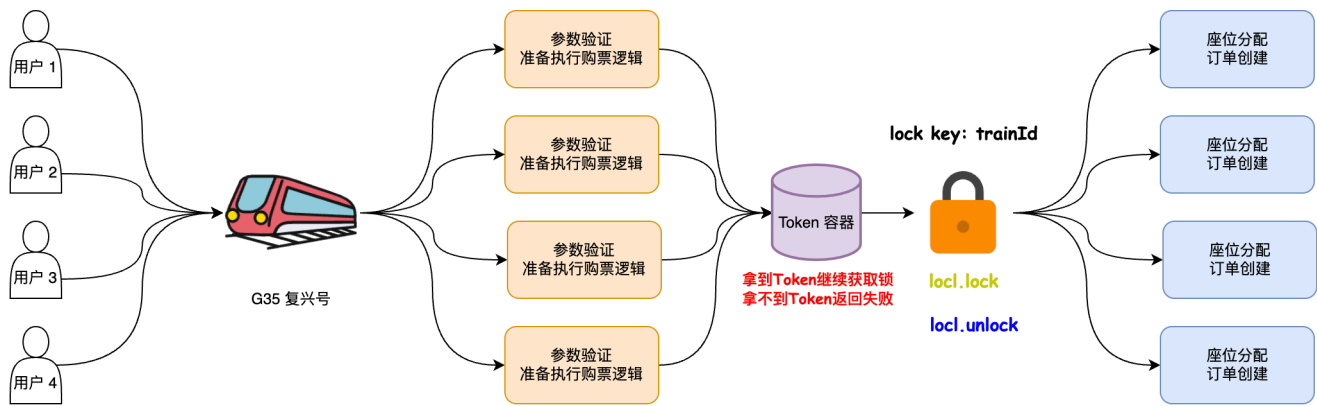
12306 如何解决限流

12306 中的限流算法类似于令牌桶，但是令牌通会以固定速率生成令牌，但是咱们这个不会，而是将没有出售的座位当作一个个令牌放到一个容器中。为了区分令牌桶，咱们下文称呼为令牌容器。

如果用户来购票，根据所选择的乘车人数量以及座位类型去令牌容器中获取，获取成功则表示余票充足，可以进入接下来的选票以及下单流程，获取失败则直接返回。

注意，令牌容器中的令牌是有限的，如果你获取到令牌后，令牌容器的数量会进行相应的减少，而这也是防止余票不超卖的原因之一。因为咱们的令牌数量和车票座位的余量是一一对应的。

加入限流流程之后，购票流程就成了这个样子，如下所示：



细心的小伙伴应该有发现，在咱们加入令牌容器这一步后，上文讲的**瞬时高并发压垮系统**问题是不是就不存在了？

之前没有令牌容器进行过滤，所有请求都能进行到分布式锁这一步，加入令牌容器后，只有少量拿到令牌的用户请求可以获取分布式锁。

12306 限流实现原理

v2 版本的购票接口地址：`org.opengoofy.index12306.biz.ticket.service.service.impl.TicketServiceImpl#purchaseTicketsV2`

大家需要关注的是从令牌容器中获取令牌方法：

`org.opengoofy.index12306.biz.ticket.service.service.handler.ticket.tokenbucket.TicketAvailabilityTokenBucket#takeToken`

其实乍一看上面代码非常复杂，但是如果拆成一个一个小部分就比较简单明了。

主要做了三件事：

1. 如果令牌容器在缓存中失效需要重新读取并放入缓存；
2. 准备执行 Lua 脚本的数据；
3. 最终的执行 Lua 脚本获取令牌。

令牌容器失效则赋值

为了避免 Redis 极端场景下的数据被删除或者失效等问题，我们这里考虑到这种情况所以先通过判断令牌容器是否存在，存在则继续，不存在则进行加载。

```
// 获取列车信息
TrainDO trainDO = distributedCache.safeGet(
    TRAIN_INFO + requestParam.getTrainId(),
    TrainDO.class,
    () -> trainMapper.selectById(requestParam.getTrainId()),
    ADVANCE_TICKET_DAY,
    TimeUnit.DAYS);
// 获取列车经停站之间的数据集合，因为一旦失效要读取整个列车的令牌并重新赋值
List<RouteDTO> routeDTOList = trainStationService
    .listTrainStationRoute(requestParam.getTrainId(), trainDO.getStartStation(), trainDO.getEndStation());
StringRedisTemplate stringRedisTemplate = (StringRedisTemplate) distributedCache.getInstance();
// 令牌容器是个 Hash 结构，组装令牌 Hash Key
String actualHashKey = TICKET_AVAILABILITY_TOKEN_BUCKET + requestParam.getTrainId();
// 判断令牌容器是否存在
Boolean hasKey = distributedCache.hasKey(actualHashKey);
// 如果令牌容器 Hash 数据结构不存在了，执行加载流程
if (!hasKey) {
    // 为了避免出现并发读写问题，所以这里通过分布式锁锁定
    RLock lock = redissonClient.getLock(String.format(LOCK_TICKET_AVAILABILITY_TOKEN_BUCKET, requestParam.getTrainId()));
    lock.lock();
    try {
```

```

// 双重判定锁，避免加载数据请求多次请求数据库
Boolean hasKeyTwo = distributedCache.hasKey(actualHashKey);
if (!hasKeyTwo) {
    List<Integer> seatTypes = VehicleTypeEnum.findSeatTypesByCode(trainDO.getTrainType());
    Map<String, String> ticketAvailabilityTokenMap = new HashMap<>();
    for (RouteDTO each : routeDTOList) {
        List<SeatTypeCountDTO> seatTypeCountDTOList = seatMapper.listSeatTypeCount(Long.parseLong(requestParam.getTrainId()),
each.getStartStation(), each.getEndStation(), seatTypes);
        for (SeatTypeCountDTO eachSeatTypeCountDTO : seatTypeCountDTOList) {
            // 组装 Hash 数据结构内部的 Key
            String buildCacheKey = StrUtil.join("_", each.getStartStation(), each.getEndStation(),
eachSeatTypeCountDTO.getSeatType());
            // 一个 Hash 结构下有很多 Key，为了避免多次网络 IO，这里组装成一个本地 Map，通过 putAll 方法请求一次 Redis
            ticketAvailabilityTokenMap.put(buildCacheKey, String.valueOf(eachSeatTypeCountDTO.getSeatCount()));
        }
    }
    // 将组装好的 Map 数据，赋值到 Redis
    stringRedisTemplate.opsForHash().putAll(TICKET_AVAILABILITY_TOKEN_BUCKET + requestParam.getTrainId(),
ticketAvailabilityTokenMap);
}
} finally {
    lock.unlock();
}
}

```

双重判定锁文章讲解：[缓存击穿之双重判定锁如何优化性能？](#)

给大家看一眼令牌算法在 Redis 中长什么样子。

令牌限流容器 Redis Key: mading:index12306-ticket-service:ticket_availability_token_bucket:1，组成结构如下：

- mading：项目中全局前缀标识 Key 标识。
- index12306-ticket-service:ticket_availability_token_bucket：令牌限流容器的 Key 标识。
- 1：列车 ID 信息。

对应的 Hash 中的元素如下，Key 就是出发站到终点站以及座位类型，对应的 Value 就是列车座位余量。

下图中的出发站到终点站之间的数据是通过[如何扣减列车座位库存](#)这种方式关联构建出来的，因为要扣减中间站点以及沿途各站的余量。

我使用的 Redis 可视化桌面终端软件：[Another Redis Desktop Manager](#)

Hash

mading:index12306-ticket-service:ticket_availability_token_bucket ✓

TTL

-1

×

✓

🗑️

🔄

</>

添加新行

ID (Total: 30)	Key ⬇	Value ⬇	输入关键字搜索
1	济南西_宁波_0	20	📄 🔍 🗑️ </>
2	济南西_宁波_2	810	📄 🔍 🗑️ </>
3	济南西_宁波_1	140	📄 🔍 🗑️ </>
4	杭州东_宁波_2	810	📄 🔍 🗑️ </>
5	杭州东_宁波_1	140	📄 🔍 🗑️ </>
6	杭州东_宁波_0	10	📄 🔍 🗑️ </>
7	济南西_杭州东_1	140	📄 🔍 🗑️ </>
8	济南西_杭州东_2	810	📄 🔍 🗑️ </>
9	济南西_杭州东_0	20	📄 🔍 🗑️ </>
10	北京南_杭州东_2	810	📄 🔍 🗑️ </>
11	北京南_杭州东_1	140	📄 🔍 🗑️ </>
12	北京南_济南西_2	810	📄 🔍 🗑️ </>
13	济南西_南京南_2	810	📄 🔍 🗑️ </>
14	济南西_南京南_0	20	📄 🔍 🗑️ </>
15	南京南_杭州东_0	20	📄 🔍 🗑️ </>
16	济南西_南京南_1	140	📄 🔍 🗑️ </>
17	北京南_济南西_1	140	📄 🔍 🗑️ </>
18	北京南_济南西_0	20	📄 🔍 🗑️ </>
19	南京南_宁波_0	20	📄 🔍 🗑️ </>
20	南京南_杭州东_2	810	📄 🔍 🗑️ </>
21	南京南_杭州东_1	140	📄 🔍 🗑️ </>
22	北京南_宁波_0	20	📄 🔍 🗑️ </>
23	北京南_杭州东_0	20	📄 🔍 🗑️ </>
24	北京南_宁波_1	140	📄 🔍 🗑️ </>

准备 Lua 脚本执行数据

执行完上面的逻辑，已经确保令牌限流容器在 Redis 中是存在的，再接下来就是准备执行获取令牌流程了。

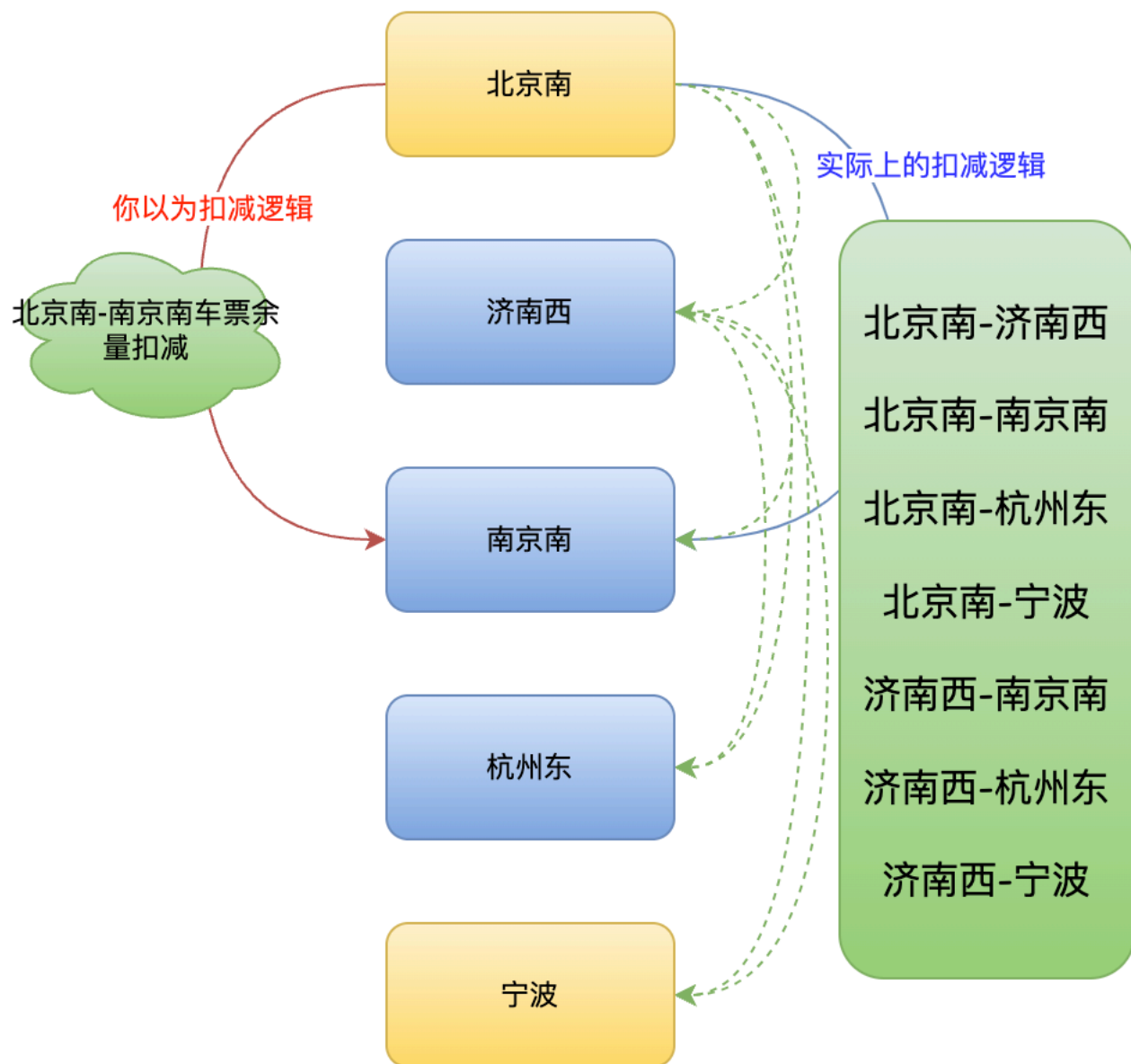
```
// 获取到 Redis 执行的 Lua 脚本
DefaultRedisScript<Long> actual = Singleton.get(LUA_TICKET_AVAILABILITY_TOKEN_BUCKET_PATH, () -> {
    DefaultRedisScript<Long> redisScript = new DefaultRedisScript<>();
    redisScript.setScriptSource(new ResourceScriptSource(new ClassPathResource(LUA_TICKET_AVAILABILITY_TOKEN_BUCKET_PATH)));
    redisScript.setResultType(Long.class);
    return redisScript;
});
// 判断 Lua 脚本不是空的，这一步基本上可以说是永远不会出错，为了避免 IDEA 波浪线才加的
```

```

Assert.notNull(actual);
// 因为购票时，一个用户可以为多个乘车人买票，而多个乘车人又能购买不同的票，所以这里需要根据座位类型进行分组
Map<Integer, Long> seatTypeCountMap = requestParam.getPassengers().stream()
    .collect(Collectors.groupingBy(PurchaseTicketPassengerDetailDTO::getSeatType, Collectors.counting()));
// 最终结构就是拆分为一个 Map，Key 是座位类型，value 是该座位类型的购票人数
JSONArray seatTypeCountArray = seatTypeCountMap.entrySet().stream()
    .map(entry -> {
        JSONObject jsonObject = new JSONObject();
        jsonObject.put("seatType", String.valueOf(entry.getKey()));
        jsonObject.put("count", String.valueOf(entry.getValue()));
        return jsonObject;
    })
    .collect(Collectors.toCollection(JSONArray::new));
// 获取需要判断扣减的站点
List<RouteDTO> takeoutRouteDTOList = trainStationService
    .listTakeoutTrainStationRoute(requestParam.getTrainId(), requestParam.getDeparture(), requestParam.getArrival());
// 用户购买的出发站点和到达站点
String luaScriptKey = StrUtil.join("_", requestParam.getDeparture(), requestParam.getArrival());

```

以 G35 复兴号高铁列车距离，购买北京南到南京南，需要扣减的站点如下所示：



更详细的站点扣减规则参考：[如何扣减列车余票](#)章节描述。

假如说，三人购票，分别是商务座两人，一等座一人，那 seatTypeCountArray 的数据结构如下：

```
[
  {
    "seatType": "0",
    "count": "2"
  },
  {
    "seatType": "1",
    "count": "1"
  }
]
```

执行 Lua 脚本获取令牌

在介绍 Lua 脚本之前，我们先看看都向 Lua 脚本输入了什么参数，介绍如下：

- actualHashKey: 存储 Redis Hash 结构的 Key 值，如上文所列 mading:index12306-ticket-service:ticket_availability_token_bucket:1。
- luaScriptKey: 用户购买的出发站点和到达站点，比如北京南_南京南。
- seatTypeCountArray: 需要扣减的座位类型以及对应数量。
- takeoutRouteDTOList♦: 需要扣减的相关列车站点。

```
Long result = stringRedisTemplate.execute(actual, Lists.newArrayList(actualHashKey, luaScriptKey),
JSON.toJSONObject(seatTypeCountArray), JSON.toJSONObject(takeoutRouteDTOList));
return result != null && Objects.equals(result, 0L);
```

最后，通过客户端调用 Redis 执行 Lua 脚本。

```
# KEYS[2] 就是咱们用户购买的出发站点和到达站点，比如北京南_南京南
local inputString = KEYS[2]
local actualKey = inputString
# 因为 Redis Key 序列化器的问题，会把 mading: 给加上
# 所以这里需要把 mading: 给删除，仅保留北京南_南京南
# actualKey 就是北京南_南京南
local colonIndex = string.find(actualKey, ":")
if colonIndex ~= nil then
    actualKey = string.sub(actualKey, colonIndex + 1)
end

# ARGV[1] 是需要扣减的座位类型以及对应数量
local jsonArrayStr = ARGV[1]
# 因为传递过来是 JSON 字符串，所以这里再序列化成对象
local jsonArray = cjson.decode(jsonArrayStr)

# 返回的 JSON 对象字符串
local result = {}
// 返回 JSON 对象字符串中的是否请求成功字段
local tokenIsNull = false
// 如果请求失败，返回 JSON 对象字符串中的请求失败座位类型和数量
local tokenIsNullSeatTypeCounts = {}

# 简单的 for 循环
for index, jsonObj in ipairs(jsonArray) do
    local seatType = tonumber(jsonObj.seatType)
    local count = tonumber(jsonObj.count)
    local actualInnerHashKey = actualKey .. "_" .. seatType
    # 判断指定座位 Token 余量是否超过购买人数
    local ticketSeatAvailabilityTokenValue = tonumber(redis.call('hget', KEYS[1], tostring(actualInnerHashKey)))
    # 如果超过那么设置 TOKEN 获取失败，以及记录失败座位类型和获取数量
    if ticketSeatAvailabilityTokenValue < count then
        tokenIsNull = true
        table.insert(tokenIsNullSeatTypeCounts, seatType .. "_" .. count)
    end
end

result['tokenIsNull'] = tokenIsNull
# 如果令牌不足则直接返回失败
if tokenIsNull then
    result['tokenIsNullSeatTypeCounts'] = tokenIsNullSeatTypeCounts
    # 序列化 JSON 字符串
    return cjson.encode(result)
end
```

```
# 通过上面的判断，已经知道令牌容器中对应的出发站点和到达站点对应的座位类型余票充足，开始进行扣减
local alongJsonArrayStr = ARGV[2]
# 因为传递过来是 JSON 字符串，所以这里再序列化成对象
local alongJsonArray = cjson.decode(alongJsonArrayStr)

# 双层 for 循环
for index, jsonObj in ipairs(jsonArray) do
    local seatType = tonumber(jsonObj.seatType)
    local count = tonumber(jsonObj.count)
    for indexTwo, alongJsonObj in ipairs(alongJsonArray) do
        local startStation = tostring(alongJsonObj.startStation)
        local endStation = tostring(alongJsonObj.endStation)
        # 开始扣减出发站点和到达站点相关的车站令牌余量
        local actualInnerHashKey = startStation .. "_" .. endStation .. "_" .. seatType
        # 扣减命令通过 hash 结构的自增命令 hincrby，因为咱们是要扣减，所以 count 前面加了个负号
        redis.call('hincrby', KEYS[1], tostring(actualInnerHashKey), -count)
    end
end

# 全部扣减完成没有异常
return cjson.encode(result)
```

最后通过 Lua 脚本返回值判断获取令牌是否成功。

```
return result == null
    ? TokenResultDTO.builder().tokenIsNull(Boolean.TRUE).build()
    : result;
```

假设我们买了两张票，商务座和一等座，示例如下：

请核对以下信息

2024-04-09（周二）G35次北京南站（09:56开）——杭州东站（04:30到）

序号	席别	票种	姓名	证件类型	证件号码
1	商务座	成人票	万重山	中国居民身份证	1101*****1955
2	一等座	成人票	金来	中国居民身份证	1101*****6713

*系统将随机为您申请席位，暂不支持自选席位

本次列车，商务座余票8,一等座余票140,二等座余票810,

返回修改 确认

如果没有令牌的情况下，这个时候返回的信息如下，_前面的 0 代表商务座，1 代表一等座，_后边代表购票人数。

```
144 String luaScriptKey = StrUtil.join( conjunction: "_", requestParam.getDeparture(), requestParam.getArrival());
145 String resultStr = stringRedisTemplate.execute(actual, Lists.newArrayList(tokenBucketHashKey, luaScriptKey),
146 TokenResultDTO result = JSON.parseObject(resultStr, TokenResultDTO.class); result: "TokenResultDTO(tokenIsNull=
return result == null = false result: "TokenResultDTO(tokenIsNull=true, tokenIsNullSeatTypeCounts=[0_1, 1_1])"
148   ∨ ∞ result = {TokenResultDTO@20225} "TokenResultDTO(tokenIsNull=true, tokenIsNullSeatTypeCounts=[0_1, 1_1])"
149   > ① tokenIsNull = {Boolean@20218} true
150   ∨ ① tokenIsNullSeatTypeCounts = {ArrayList@27675} size = 2
151   > 0 = "0_1"
152   > 1 = "1_1"
153 /**
154  * 回滚列车余票
155  *
Set value F2 Create renderer
```

...warning 很多同学对令牌容器表示质疑，为什么不能用余票缓存充当限流？首先说明哈，如果你看了下面的文章觉得令牌限流比较复杂，那么可以说使用余票缓存充当限流，**因为令牌限流是为了考虑极端情况。**

详情查看：[高并发库存扣减为什么需要令牌限流？](#)

...

分布式锁之公平锁

什么是公平锁

公平锁（Fair Lock）是一种锁的获取策略，它按照请求的先后顺序分配锁资源，以实现公平性和可预测性。

当多个线程竞争同一个公平锁时，如果锁当前是可用状态，那么锁将立即分配给等待时间最长的线程，即最先请求锁的线程。如果锁当前被其他线程持有，那么新的线程请求锁时会被放入一个等待队列中，按照请求顺序排队等待锁的释放。

公平锁的主要特点是：

1. 公平性：公平锁确保锁的获取按照请求的先后顺序进行，即先到先得的原则。这样可以避免某些线程长时间等待锁资源而导致饥饿问题，公平地分配锁资源给所有等待的线程。
2. 可预测性：由于按照请求的先后顺序分配锁资源，公平锁的行为是可预测的。每个线程都可以预期在其请求锁后的一段时间内获得锁，而不会受到其他线程的插队或优先级干扰。

公平锁的实现需要维护一个等待队列，记录等待锁资源的线程，并按照请求顺序分配锁。这可能会增加一些开销，包括锁的管理和线程调度的开销。在高并发情况下，公平锁的性能可能相对较低，但可以提供公平性和可预测性的保证。

公平锁适用于对锁资源的分配有严格要求的场景，例如资源竞争激烈、对各个线程的公平性要求较高的情况。

上面已经说了，要解决买票严格按照顺序执行，就需要使用公平锁分配锁，确保较早请求的线程先获得锁。

如何改造公平锁

从默认的非公平锁切换到公平锁上，代码变更非常简单，只需要修改获取锁方法名为 getFairLock 即可。

```
RLock lock = redissonClient.getFairLock(lockKey);
lock.lock();
try {
    return executePurchaseTickets(requestParam);
} finally {
    lock.unlock();
}
```

非公平锁一定不好么

相对于非公平锁，公平锁在某些方面可能存在以下缺点：

1. 性能降低：公平锁需要维护一个等待队列，按照请求的顺序分配锁资源。这会增加锁的管理开销和线程调度的开销，可能导致性能下降，尤其在高并发场景下。
2. 竞争增加：由于公平锁要按照请求的顺序分配锁资源，当一个线程释放锁时，需要通知等待队列中的下一个线程获取锁。这会引起更多的竞争和上下文切换，从而降低系统的吞吐量。

再额外提醒大家一句，技术设计中不存在“银弹”。选择技术选型往往会有得失，多方面权衡后选择一个适合项目的使用即可。

分布式锁进阶

分布式锁 v1 架构开发完成后，星球沟通群中有位同学提了一个关于分布式锁优化的观点，我仔细思考后觉得说的很有道理，接下来我们就看看是否可行呢。

这里多说一句，如果大家加了星球，一定要加我微信：magestacks，备注星球，拉大家进星球 VIP 群。

拿个 offer-开源&项目实战 (500)

@马丁@ 优先星球提问 马哥类似于这种有加锁逻辑的，因为都是都是集群化部署，是否考虑封装下加锁逻辑，线程先去竞争单个服务的内部锁，竞争成功再去竞争分布式锁，从而减少redis的压力

构建本地分布式多重锁

如果要是实现上面这位同学说的话，我们 v2 接口的实现逻辑需要再次重构下，从单分布式锁的获取变为多种锁的组合获取。

```
private final ConcurrentHashMap<String, ReentrantLock> localLockMap = new ConcurrentHashMap<>();

@Override
@Transactional(rollbackFor = Throwable.class)
public TicketPurchaseRespDTO purchaseTicketsV2(PurchaseTicketReqDTO requestParam) {
    // .....
    // 构建锁唯一 Key
    String lockKey = environment.resolvePlaceholders(String.format(LOCK_PURCHASE_TICKETS, requestParam.getTrainId()));
    // 根据锁唯一 Key 获取本地锁，通过 ConcurrentHashMap 保证并发读写数据安全
    ReentrantLock localLock = localLockMap.computeIfAbsent(lockKey, key -> new ReentrantLock(true));
    // 先获取本地公平锁，因为咱们上面创建锁指定了公平模式 new ReentrantLock(true)
    localLock.lock();
    try {
        // 获取到本地公平锁后，开始获取分布式公平锁
        RLock lock = redissonClient.getFairLock(lockKey);
        lock.lock();
    }
```

```

        try {
            // 执行购票流程
            return executePurchaseTickets(requestParam);
        } finally {
            // 释放分布式公平锁
            lock.unlock();
        }
    } finally {
        // 释放本地公平锁
        localLock.unlock();
    }
}

```

从实现咱们上述功能来说，这个代码已经没问题了。但是，仔细思考下，是否还有一些潜在逻辑是没考虑到的？

本地锁内存安全思考

上面这个程序安全么？在看到这里时，大家思考下。

结论是不安全的，可能会有内存溢出的风险。问题就出在本地锁存储容器上。

我们通过 `ConcurrentHashMap` 存储每个列车的本地锁，作为申请分布式锁之前的一层性能挡板，隔绝无效流量请求 Redis。但是大家发现没有，这个 `ConcurrentHashMap` 是只能存储，但是没有任何过期策略。这样会导致一个问题就是应用长时间不发布，越来越多的列车数据存储在容器中，直到内存溢出为止。

怎么实现一个线程安全以及内存安全的本地锁容器？伪代码如下，大家仅作为参考即可。

通过 `Caffeine` 创建本地安全锁容器，`Caffeine` 的 `expireAfterWrite` 方法代表，放入元素过期的时间是什么。比如咱们以下案例中配置的一天过期，代表一个列车的本地公平锁创建一天后失效。

```

private final Cache<String, ReentrantLock> localLockMap = Caffeine.newBuilder()
    .expireAfterWrite(1, TimeUnit.DAYS)
    .build();

@Override
@Transactional(rollbackFor = Throwable.class)
public TicketPurchaseRespDTO purchaseTicketsV2(PurchaseTicketReqDTO requestParam) {
    // .....
    String lockKey = environment.resolvePlaceholders(String.format(LOCK_PURCHASE_TICKETS, requestParam.getTrainId()));
    // 通过原子操作新增 ReentrantLock，如果 localLockMap 存在则返回，不存在则新增
    // 可能有同学问：为什么不直接使用 putIfAbsent API 写入锁？
    // 使用 putIfAbsent 时，必须预先创建锁对象，无论该锁是否已存在。若多个线程同时尝试插入同一个 lockKey，会导致大量冗余的 ReentrantLock 对象
    // 被创建（即使最终被丢弃），增加 GC 压力
    // computeIfAbsent 仅在键不存在时按需创建锁对象，避免无效的临时对象生成，显著减少内存占用和 GC 开销
    ReentrantLock localLock = localLockMap.asMap().computeIfAbsent(
        lockKey,
        k -> new ReentrantLock(true)
    );
    localLock.lock();
    try {
        RLock lock = redissonClient.getFairLock(lockKey);
        lock.lock();
        try {
            return executePurchaseTickets(requestParam);
        } finally {
            lock.unlock();
        }
    } finally {
        localLock.unlock();
    }
}

```

```
}  
}
```

令牌限流算法后本地锁是否有必要

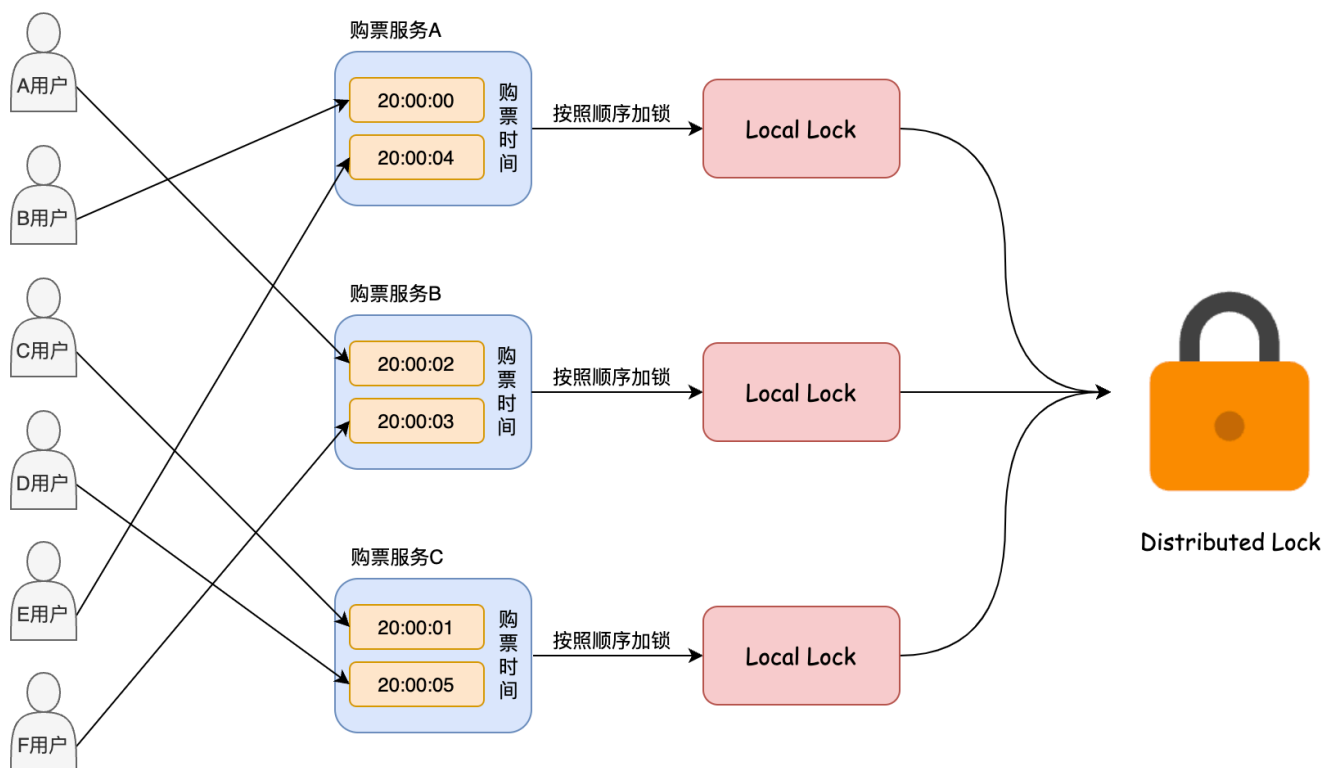
如果是 v1 版本购票接口请款下，本地公平锁肯定是要的。但咱们添加了令牌限流算法后，这个本地锁就变的可有可无了，因为经过令牌限流算法过滤后，真正能打到 Redis 分布式锁的请求就已经百中无一了。如果大家想进一步减少缓存的压力，可以加。

在设计高并发情况下，应该避免一切无效代码，要尽力提升接口性能和吞吐量。既然本地锁能提升性能，为什么我没和大家说一定要加？

具体原因请看下一章节。

本地锁还能保证购票顺序么

如果加了本地锁还能保证购票顺序么？通过下面这张图得知，答案是否定的。



如果加入本地锁后，全局的购票顺序将变为局部顺序。这将带来一个折衷的问题：是要通过使用分布式锁来确保全局有序性，还是使用本地锁和分布式锁的组合来保持局部有序性？

以下是使用本地锁+分布式锁组合的情况：

- 当购票人数非常多时，为了减轻缓存的压力，可以适当牺牲一定的全局有序性。

以下是使用分布式锁的情况：

- 购票人数不是很多时，为了保障用户的座位和购票顺序，需要确保全局有序性。在这种情况下，由于缓存基本上没有压力，使用分布式锁可以保证购票的正确性和顺序性。

需要根据具体的业务需求和系统规模来选择合适的锁策略。在购票人数较多且缓存压力较大时，本地锁和分布式锁的组合可以在一定程度上平衡性能和购票顺序。而在购票人数较少且重视购票顺序的情况下，使用分布式锁可以确保全局有序性。

优化业务加锁粒度

上文说过，[用户购票响应慢](#)的问题，经过上面几种方案，已经一定程度提升了很多响应速度。

什么是加锁粒度

还有没有能再优化的技术点？还有一个可以优化的点，那就是本地锁和分布式锁的粒度。咱们目前是锁定的列车 ID 值，是不是可以锁定到具体的列车座位类型？

可以是可以，但是这里有一个问题，用户购票时是可以为多个乘车人选择不同的座位，这种如何解决？

列车信息（以下余票信息仅供参考）

2023-08-31（周四）G1063次 北京南站（09:28开）—天津南站（10:02到）

商务座（¥235.0）8张票 一等座（¥104.0）有票 二等座（¥63.0）有票

*显示的卧铺票价均为上铺票价，供您参考。具体票价以您确认支付时实际购买的铺别票价为准。

乘客信息（填写说明）

输入乘客姓名

乘车人

序号	票种	席别	姓名	证件类型	证件号码
1	成人票	二等座（¥63.0）		中国居民身份证	
2	成人票	一等座（¥104.0）		中国居民身份证	

中国铁路保险

乘意相伴 安心出行

乘意险重磅升级 保障范围更全面

提交订单表示已阅读并同意 《国铁集团铁路旅客运输规程》 《服务条款》

上一步

提交订单

如何调整业务锁粒度

当用户为多个乘车人购票选择了多个不同座位类型时，我们可以加多把锁，锁全部获取到才可以执行购票流程。

```
private final Cache<String, ReentrantLock> localLockMap = Caffeine.newBuilder()
    .expireAfterWrite(1, TimeUnit.DAYS)
    .build();

@Override
@Transactional(rollbackFor = Throwable.class)
public TicketPurchaseRespDTO purchaseTicketsV2(PurchaseTicketReqDTO requestParam) {
    // .....
    // 存储本次请求需要获取的本地锁的集合
    List<ReentrantLock> localLockList = new ArrayList<>();
    // 存储本次请求需要获取的分布式锁的集合
    List<RLock> distributedLockList = new ArrayList<>();
    // 按照座位类型进行分组
    Map<Integer, List<PurchaseTicketPassengerDetailDTO>> seatTypeMap = requestParam.getPassengers().stream()
        .collect(Collectors.groupingBy(PurchaseTicketPassengerDetailDTO::getSeatType));
    seatTypeMap.forEach((seatType, count) -> {
        // 构建锁 Key，相比较上个版本，增加了座位类型
        String lockKey = environment.resolvePlaceholders(String.format(LOCK_PURCHASE_TICKETS_V2, requestParam.getTrainId(), seatType));
        ReentrantLock localLock = localLockMap.getIfPresent(lockKey);
```



```

        if (localLock == null) {
            synchronized (TicketService.class) {
                if ((localLock = localLockMap.getIfPresent(lockKey)) == null) {
                    localLock = new ReentrantLock(true);
                    localLockMap.put(lockKey, localLock);
                }
            }
        }
        // 添加到本地锁集合
        localLockList.add(localLock);
        RLock distributedLock = redissonClient.getFairLock(lockKey);
        // 添加到分布式锁集合
        distributedLockList.add(distributedLock);
    });
    try {
        // 循环请求本地锁
        localLockList.forEach(ReentrantLock::lock);
        // 循环请求分布式锁
        distributedLockList.forEach(RLock::lock);
        return executePurchaseTickets(requestParam);
    } finally {
        // 释放本地锁
        localLockList.forEach(localLock -> {
            try {
                localLock.unlock();
            } catch (Throwable ignored) {
            }
        });
        // 释放分布式锁
        distributedLockList.forEach(distributedLock -> {
            try {
                distributedLock.unlock();
            } catch (Throwable ignored) {
            }
        });
    }
}
}

```

为什么要加这个锁优化呢？大家设想一下，高峰期比如五一、国庆以及春节这些大的节日，购票用户大多是自己或者情侣或者家人购买车票，大家是不是很少会去选择一批乘车人购买不同座位类型的票？那这样的话，咱们的这个优化，最好的情况下，以高铁举例，能提高 300% 的性能，因为高铁有三种类型座位。

调整后有什么问题

举个例子，张三购买了三个座，分别是商务座、一等座以及二等座，李四购买了二等座，按上述的流程来看，张三需要获取六把锁（本地和分布式各三把锁），李四需要获取两把锁。

假设两个同时请求，张三刚获取了商务座和一等座的锁，因为并发的缘故，接下来李四获取到了二等座锁，这个时候张三哪怕已经获取到了两个座位的锁，也只能等待李四执行完座位分配后再执行。**因为获取锁是顺序的，所以不存在死锁情况。**

总结下上面的描述，一共有两个执行过程中可能出现的问题：

- 性能问题：本来用户购票只需要请求一次锁，现在却因为乘车人座位的多少，而决定了会请求多加几次的锁，存在一些性能问题。
- 执行时机：如上所述，如果用户购买的座位过多，会被其他购买了单个座位的用户进行插队。

综上所述，咱们锁粒度优化更多是为了海量并发场景下的抢票场景，这种场景下，用户一般是不会选择多个座位的。所以，咱们 v2 接口加上了这个设计。

原文: <<https://www.yuque.com/magestack/12306/ov3u6lpgartx0mst>>